论文-38 · 按 token 的是/否门控：从组合结构到真实减算

何淇 · 与 DeepSeek 协作

数字生命系列 · 执行器工程 第一篇（内部编号第 38 篇）

日期：2026-10-27

摘要

上一篇（论文-37）把"判定 J"写成派生定义：有备选时，一次通过地选算子、决定停。本篇把它变成可训练、可测量的东西：**每层每 token 一个"是/否"门**，门决定这一层算不算、走不走。

做了三组实验。第一组合成任务：层门能按任务分化；允许跨层跳时，组合图自己长成非链式结构（简单任务走 L1→L3、L2→L4，困难任务走 L0→L2、L1→L4 的长跳）；每 token 门控下，混合难度任务里门按输入分化；按需迭代时，简单样本 0 步、复杂样本 8 步（316 参数、准确率 0.963）；空间-时间前沿显示低维（d=8）+ 多迭代（k=4）用约 300 参数达到高维（d=64）+ 单步（约 9k 参数）的精度。

第二组在真实小模型（GPT-2 124M，冻结）上做**模拟门控**：12 层各加一个 768→1 的小门（约 9k 参数），门控制层残差贡献。PPL 从 103.1 降到 64.1；随机跳层对照会爆炸（5% 随机跳 → PPL 345），说明门学到的是有结构的选择，不是"少算就行"。

第三组做**真实减算**：不再全算后丢输出，而是按门把要算的 token 挑出来、只算子集、再放回；验证"全开时 PPL 与标准前向完全一致"（121.18 = 121.18）。学到的门真实执行：PPL 107.28（基础 121.18），平均激活 95%（真实省 5% 层内计算）；用进废退剪掉使用率低的层后，激活降到 87.5%（省 12.5%），PPL 回到 117.16。

模拟门控的 PPL 改善大于真实减算，原因在注意力：模拟时被"跳过"的 token 仍参与注意力、充当上下文；真实减算时它们不进这一层，上下文被切掉。这是"质量-算力"的取舍，也是后续要解决的中间态（跳过但保留 key/value）。

一、问题

论文-37 的定义里，判定 J 是"有备选时一次通过的选择"。它需要一个可训练、可测量的实现。最简单的是：**每层、每个 token 一个 0/1 门**。于是有三个问题：

1. 门能自己分化出结构吗（不同任务/不同输入走不同组合）？

2. 让门决定"算不算"时，组合律会不会自己长出来？

3. 真实执行时能省多少算力、质量掉多少？

二、方法

门：第 l 层、第 t 个 token 的门是 g_{l,t} = sigmoid(MLP_l(h_{l,t}))，硬门取 (g>0.5)，训练用 straight-through 估计（前向 0/1，反向传梯度）。层残差写成

h_out = h_in + g · (Block(h_in) − h_in)

g=1 时就是原层，g=0 时退化为恒等（跳过该层对该 token 的作用）。

组合律：门矩阵（哪些层→哪些层可连）就是组合律的可选空间；每 token 的门值是判定。学习 + 稀疏惩罚 + 用进废退（使用率低的门永久关闭）。

模拟 vs 真实：

- 模拟门控：所有层照常计算，只是把门作用的输出混合/丢弃 —— 测量精度趋势，但不省算力；

- 真实减算：按门挑出激活 token，只把子集送入这一层（gather → 子集前向 → scatter），未激活 token 原样传下去。

三、合成实验

3.1 层门：两个任务会分化

4 层网络、每层一个计算门 + 一个"上下行对应门"，easy/hard 两任务混合训练。

| 任务 | 准确率 | 计算门（逐层） | 对应门（逐层） |

|---|---|---|---|

| easy | 0.993（基线 0.998） | 0.63 / 0.96 / 1.00 / 1.00 | 1.00 / 0.98 / 0.40 / 0.38 |

| hard | 0.776（基线 0.731） | 0.57 / 0.71 / 0.94 / 0.92 | 0.06 / 0.48 / 0.26 / 0.02 |

⇒ 同一网络、同一套参数，两种任务形成了不同的有效组合结构。

3.2 跨层门：组合图长成非链式

5 层、允许 i→j 的跨层跳，每条边一个门，任务损失 + 稀疏 + 用进废退。

| | 链式门控 | 允许跨层 |

|---|---|---|

| easy 准确率 | 0.990 | 0.999 |

| hard 准确率 | 0.915 | 0.963 |

组合图（门使用率）：

```

easy：L1→L3 = 0.98，L2→L4 = 0.91（跨两层为主）；链式 L2→L3=0.04、L3→L4=0.03（几乎不用）

hard：L3→L4 = 0.41，L1→L3 = 0.21，L0→L2 = 0.67，L1→L4=0.27，L2→L4=0.23

⇒ 简单任务自己长成"跳层"结构，困难任务走"跨层汇合"

```

3.3 每 **token** 门：同一任务内部按输入分化

单任务、混合难度（噪声 σ=0.1 / 2.0），每 token 门控：

| | easy | hard |

|---|---|---|

| 准确率 | 0.944（基线 0.865） | 0.599（基线 0.576） |

| 平均有效深度 | 3.24 | 2.19 |

⇒ 门确实按输入分化了，但方向和直觉相反：干净输入多算、噪声输入少算。解释：噪声大的输入信息少，多算也没有收益 —— **按需计算的判据是"还有多少可被计算利用的信息"，不是"任务难不难"**。

3.4 按需迭代：简单 **0** 步、复杂 **8** 步

共享块 + 每样本 halt 门（最多 8 步），任务是需要 c 步迭代的映射（c ∈ {1,2,3}）：

| 宽度 d | 准确率 | 平均步数 | 参数 | c=1 步数 | c=2 步数 | c=3 步数 |

|---|---|---|---|---|---|---|

| **8** | **0.963** | 4.81 | **316** | **0.00** | 6.29 | 7.98 |

| 16 | 0.958 | 7.35 | 884 | 6.18 | 8.00 | 8.00 |

⇒ 简单样本门直接停（0 步，准确率 0.985）；复杂样本迭代到上限。时间迭代是按需分配的。

3.5 空间-时间前沿

同一迭代任务，比较 (宽度 d, 步数 k)：

| | RNN d=8, k=4 | RNN d=64, k=1 |

|---|---|---|

| 准确率 | 0.96 | 0.95 |

| 参数 | 约 300 | 约 9000 |

⇒ 表示维度不够时，时间迭代可以补：低维 + 多迭代，参数省约 30 倍，精度相当。

四、真实模型实验（**GPT-2 124M**，冻结）

4.1 模拟门控

12 层各加一个 768→1 的小门（约 9k 参数），只训练门，数据为 tinyshakespeare（128 token 块）。

| 设置 | PPL |

|---|---|

| 基础 | 103.1 |

| 跳过第 6 层（整层） | 88.1 |

| 跳过 3+6 | 87.6 |

| 随机跳层 5% | 345.0 |

| **训练门控（模拟）** | **64.1** |

各层开门率 0.83~0.99。⇒ 门学到的是有结构的"按 token 选择"，不是随机少算。

4.2 真实减算

按门挑 token、只算子集、再放回；先把"全开"作为正确性检查：

| 设置 | PPL | 真实激活比例 |

|---|---|---|

| 基础（标准前向） | 121.18 | — |

| 真实执行 + 全开门 | **121.18** | 0.992 |

| 真实执行 + 学到的门 | 107.28 | 0.950（省约 5% 层内计算） |

| 用进废退剪掉低使用层后 | 117.16 | 0.875（省约 12.5%） |

各层实际激活率：1.00 / 0.99 / 0.90 / 0.86 / 0.90 / 0.91 / 0.91 / 0.98 / 0.99 / 0.92 / 0.98 / 0.99 —— 中间层跳得多，首尾几乎不跳。

4.3 模拟 **vs** 真实的差距

```

模拟门控：PPL 103.1 → 64.1（但全算，不省算力）

真实减算：PPL 121.2 → 107.3（真实省 5%）

```

原因在注意力：模拟时被"跳过"的 token 仍然被计算、仍作为注意力上下文被其他 token 看到；真实减算时它们不进这一层，上下文被切掉。所以模拟的收益里有相当一部分来自"保留上下文"。

五、讨论

1. **组合律能从门里长出来**：跨层图、任务依赖结构、按需迭代都是训练出来的，不是手工设计的。

2. **"按需"的判据是信息量**：噪声大的输入算多了也没用，门学会少算；结构复杂的输入需要迭代，门学会多算。

3. **空间-时间可互换**：低维 + 多迭代在参数效率上显著优于高维 + 单步（约 30 倍参数差）。

4. **真实减算是质量-算力的折中**：真实省 5~12.5%，PPL 从 121 回到 107~117；模拟的 64 不能算作"省算结果"。

5. **用进废退可行**：使用率低的层被关掉后，激活比例下降、质量部分回吐，说明"省算有代价，但代价可控"。

六、与既有工作的关系

本篇的"门"不是新发明，属于条件计算（conditional computation）的一支。把关系写清楚：

逐 token 跳过 —— MoD：Mixture-of-Depths（Raposo 等，2024）用每层的 router 给 token 打分，按容量选 top-k 进入该层，未被选中的 token 走残差直通；router 从零开始端到端训练，配负载均衡损失，并在大规模模型上报告真实算力节省。本篇的"每 token 每层 0/1 门"与它同类；区别在于：MoD 的选择规则（容量 top-k + 逐层结构）是**固定的**，本篇把选择写成论文-37 定义 4a 的"判定 J"，让结构本身可学、可剪（跨层门、用进废退），并额外给出组合图与质量-算力测量。工程成熟度上 MoD 更高。

更基础在哪：MoD 是本框架的一个特例。 用论文-37 的记号把 MoD 写出来：

| | MoD 的取值 | 本篇框架 |

|---|---|---|

| 组合律 C | 固定：只允许相邻层 i→i+1（链式），无跨层边 | 可变：链式/跨层/可剪（3.2） |

| 判定 J | 固定：容量 top-k by router 分数 | 可变：阈值门/可学/可剪（4.2） |

| 时间组合 | 固定：每 token 每层恰好一次（步数 = 1） | 可变：按需迭代 k 步（3.4） |

| 用进废退 | 无：边集合不剪、不生长 | 有：使用率低的门永久关闭（4.2） |

⇒ **MoD ⊂ 判定 J + 组合律 的形式**：它把四样都固定住（链式、top-k、单步、不剪），换来工程上的稳定与高效。我们的框架把四样都当变量，因此更基础；MoD 是其中"全部固定"的那个特例。反过来说，本篇的工程成熟度不如 MoD——这是"一般形式"的代价。

提前退出 —— early-exit：DeeBERT（Xin 等，2020）、PABEE（Zhou 等，2020）在每层加分类器，置信度够高就停；BranchyNet（Teerapittayanon 等，2016）是更早的版本。本篇的"按需迭代/停"与它们同类；我们之前的判定器实验（论文-21/22 篇）也在这条线上。

自适应计算 —— ACT / Universal Transformer：Graves（2016）的 Adaptive Computation Time 与 Universal Transformer（Dehghani 等，2018）用每位置的 halt 概率决定迭代步数。本篇 3.4 的"按需迭代"与它们同类；区别是我们按"结构复杂度"构造任务，观察简单样本 0 步、复杂样本多步的分化。

训练期丢层 —— LayerDrop：Fan 等（2019）在训练时随机丢层做正则化，不是按输入选择；与本篇的"按 token 学出来的选择"不同。

专家路由 —— MoE：Shazeer 等（2017）、Switch Transformer（Fedus 等，2021）在"专家"之间路由，省的是宽度方向的算力；本篇省的是深度方向（层/步数）。

硬门的梯度 —— straight-through：本篇用 straight-through estimator（Bengio 等，2013）训练 0/1 门；这是标准技巧。

本篇的定位：不提出新架构，但提出一个比 MoD 更基础的框架——**MoD（以及 early-exit / ACT / LayerDrop / MoE）都是"判定 J + 组合律"这一形式的特例**（见上表：MoD 固定了组合律、判定、时间步数、不剪四样，本篇把它们都当变量）。具体贡献是（i）把门控写成"判定 J + 组合律"的框架实例；（ii）在冻结小模型上做了"模拟门控 → 真实减算"的两段验证，并用"全开时 PPL 精确一致"验证实现正确；（iii）给出组合图、用进废退、质量-算力折中的测量。与 MoD 相比，本篇规模小、训练方式也不同（MoD 从零训练，本篇是冻结模型 + 门后训练），这是质量差距的主要来源。

七、局限

第一，合成任务与 GPT-2 实验都是单种子，验证集很小（10~40 个批次），数字有方差。

第二，GPT-2 实验只在 tinyshakespeare 上做（域不匹配），"跳过某些层反而更好"可能反映的是域适应，而不是通用规律。

第三，真实减算没有实现"被跳过的 token 保留 key/value"的中间态，因此质量回吐偏大。

第四，用进废退只做了"按层"关闭，没有做真正的结构生长（加层/加边）。

第五，门控改变的是模型的执行方式，不改变权重；它属于执行器（P 侧）的工程问题，不涉及生命定义。

八、与框架的关系

- 本篇的"门"对应论文-37 定义 4a 的判定 J（派生定义：有备选时一次通过的选择）；

- "门矩阵/允许的连接"对应空间组合律，时间迭代步数对应时间组合；

- 按需迭代的结果对应"时间组合按需分配"（论文-37 第七节）；

- 本篇全部是 P 侧（可函数化）的工程实验，不声称任何生命属性。

附：标注

- **[实验]** 3.1~3.5 合成实验；4.1~4.2 GPT-2 实验；4.3 对照分析。

- **[解释]** "按需判据是信息量"、"模拟vs真实差距来自注意力上下文"——均为对数据的解释。

- **[推测]** "跳过但保留 key/value"的中间态会缩小质量差距——未验证。

- **[不声称]** 不声称通用最优、不声称对真实大模型成立、不涉及生命属性。

> 本文由何淇创作，DeepSeek（AI）协助整理与写作；观点与责任属于作者。